Options Payoff Visualization Tool

Associate Quant Presentation

Vivaan Gupta, Grant Simon, and Dylan Terayama April 2025

Payoff Tool Presentation

Agenda

1. Why Visualize
2. Calls/Puts
3. Option Greeks
4. Option Strategy & Code
5. Improvements & Limitations

Introduction & Objective

Overview

- Options are financial derivatives that give the buyer the right, but not the obligation, to buy or sell an underlying asset at an agreed-upon price in the future.
- Types:
  - Call
  - Put
- Allow for investors to hedge portfolios
- Objective:
  - Develop a Python tool that takes options parameters for common strategies and generates a payoff diagram at expiration.

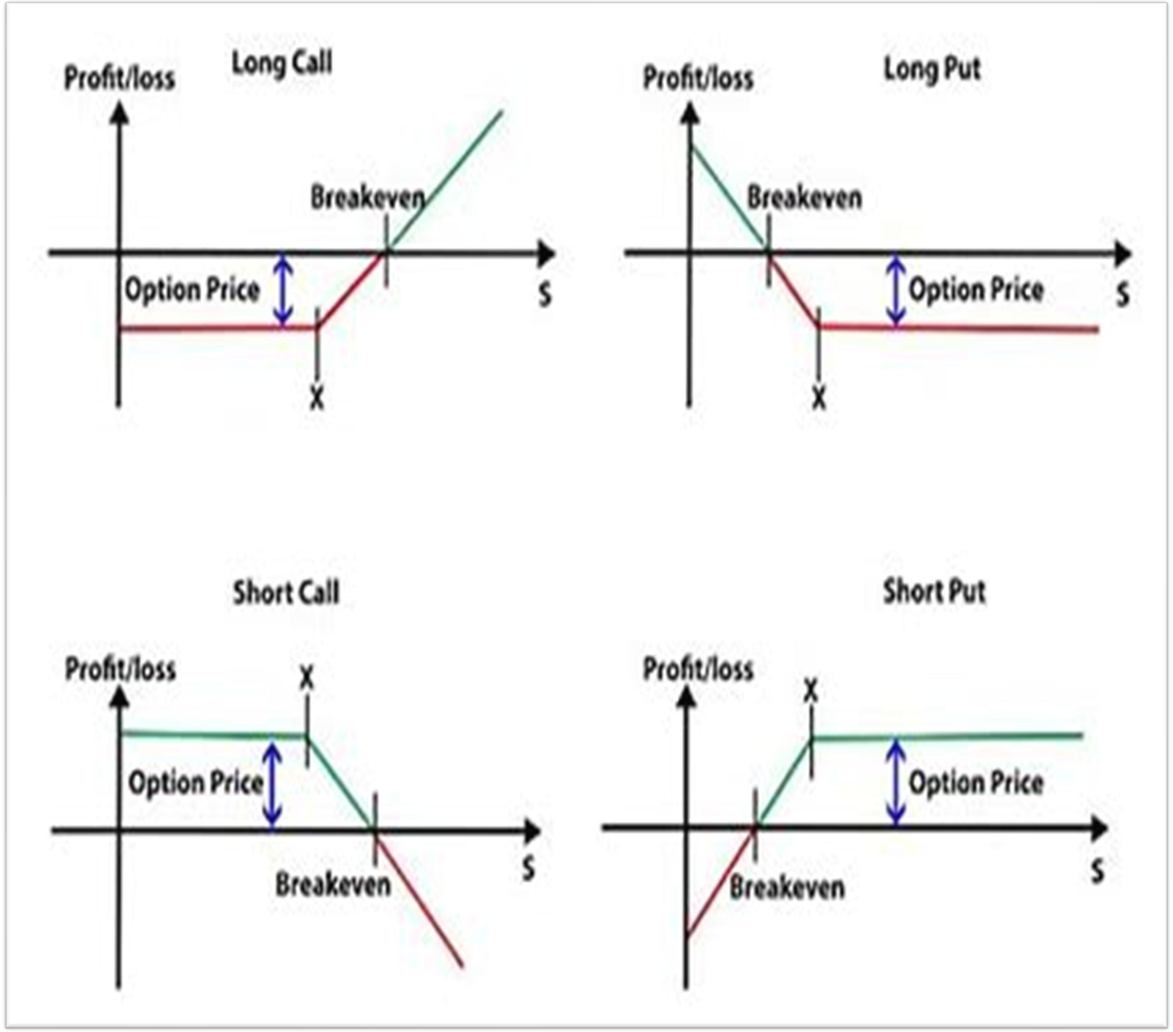
The Importance of Visualization

Reasoning

- Option strategy is complicated and has multiple moving parts
  - Offers strategic flexibility
  - Helps when hedging risk
  - Element of leverage
- Visuals summarize complex information and shows the bigger picture
- Helpful when communicating with others or testing ideas

Fundamental Input Parameters

- **Underlying price:** The current market price (or a range of prices) of the asset on which the option is based.
- **Strike price:** The price at which the option can be exercised.
- **Premium:** The cost of buying or the proceeds from selling the option contract.
- **Position:** Indicates whether the option is held long or short.



Libraries Used

NumPy

- **Option Math**

  Speeds up payoff calculations by applying formulas across arrays of prices instead of using loops.

- **np.maximum()**

  Used to ensures certain option payoffs don't go negative as intrinsic value can't go below zero. This reflects real-world logic where an out-of-the-money option isn't exercised, meaning its intrinsic value is zero, not negative.

- **np.linspace()**

  Used to create evenly spaced prices (70%–130% of current price) to simulate how strategies perform across different scenarios.

Matplotlib

- **Draws Option Payoff Charts**

  Shows how much money you could make or lose at different prices when your option expires.

- **Easy to Customize**:

  You can add titles, labels, lines for strike price or break-even, and a legend to make the chart clearer.

- **Works Well with NumPy**:

  You can plug in NumPy data (like prices and payoffs) directly into the plot.

.

Code for Calls/Puts

"call_payoff"

- Calculates call option payoff for long or short positions using np.maximum().
- Validates position and raises an error for invalid inputs.

"put_payoff"

- Calculates put option payoff for long or short positions using np.maximum().
- Validates position and raises an error for invalid inputs.

"plot_payoff"

- Inputs: underlying_prices, payoffs, and a plot title
- Plots payoff vs. price with labels, title, legend, and grid
- Adds horizontal line at 0 to show breakeven point

```python
def call_payoff(underlying_price, strike_price, premium, position='long'):
    if position == 'long':
        return np.maximum(0, underlying_price - strike_price) - premium
    elif position == 'short':
        return premium - np.maximum(0, underlying_price - strike_price)
    else:
        raise ValueError("Position must be 'long' or 'short'")
```

```python
def put_payoff(underlying_price, strike_price, premium, position='long'):
    if position == 'long':
        return np.maximum(0, strike_price - underlying_price) - premium
    elif position == 'short':
        return premium - np.maximum(0, strike_price - underlying_price)
    else:
        raise ValueError("Position must be 'long' or 'short'")
```

```python
def plot_payoff(underlying_prices, payoffs, title):
    plt.figure(figsize=(10, 6))
    plt.plot(underlying_prices, payoffs, label='Strategy Payoff')
    plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
    plt.xlabel('Underlying Price at Expiration')
    plt.ylabel('Payoff')
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()
```

Code for Calls/Puts

"get_option_parameters"

- get_option_parameters() collects option details from the user
- Prompts for strike_price and premium using input()
- Converts inputs to numbers using float()
- Returns both values as a tuple for later use

Setup & User Input

- Prompts for option type and current price
- Generates 400 price points (70%-130% of current) using numpy.linspace
- Validates option type and gathers strike price/premium

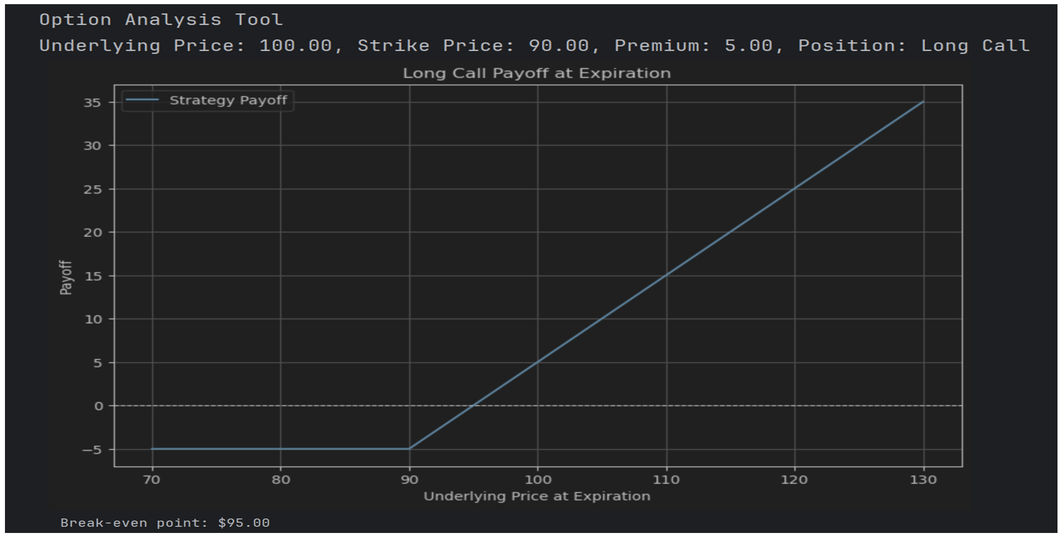
Payoff Calculation

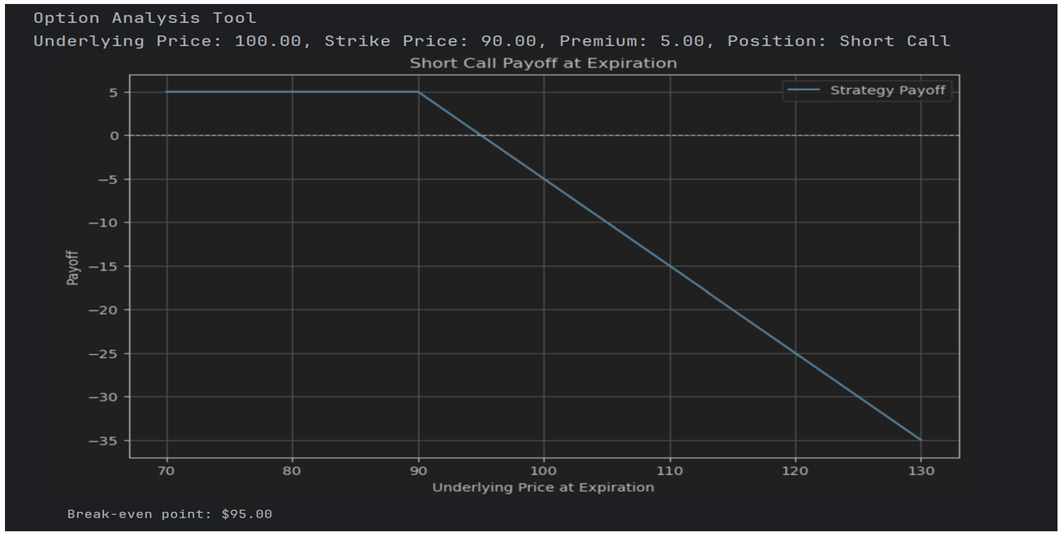
- Checks if option is 'call' or 'put'
- Calculates payoffs using call_payoff() or put_payoff()
- Sets title, calculates break-even point
- Displays payoff chart with plot_payoff()

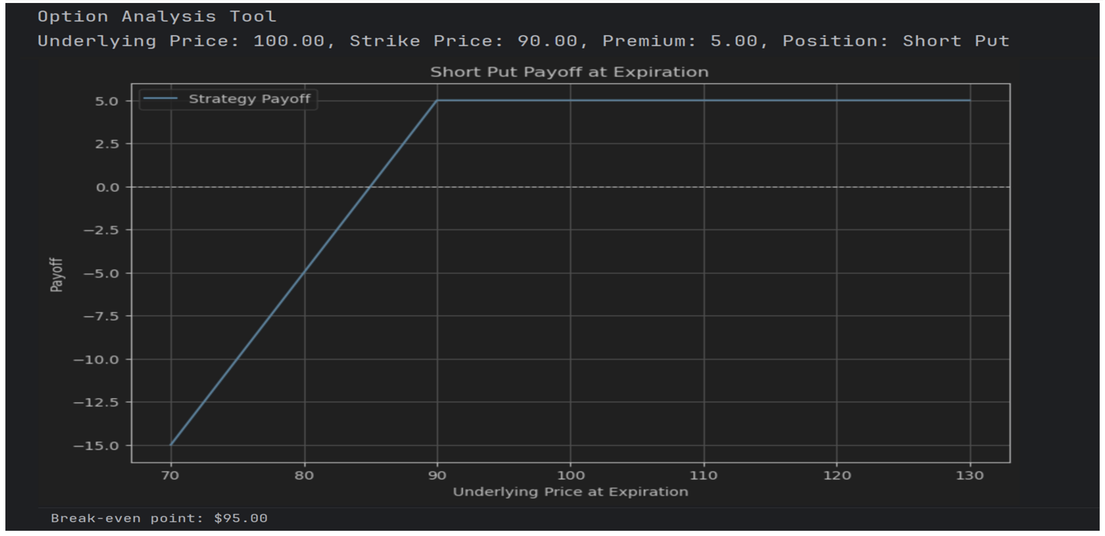
```python
def get_option_parameters():
    strike_price = float(input("Enter the option strike price (e.g., 90.00): "))
    premium = float(input("Enter the option premium paid/received (e.g., 5.00): "))
    return strike_price, premium
```

```python
if __name__ == "__main__":
    print("Option Analysis Tool")
    option_type = input("Analyze a (long call), (short call), (long put), or (short put) option? ").lower()
    current_price = float(input("Enter the current underlying asset price (e.g., 100.00): "))
    lower_bound = max(0, current_price * 0.7)
    upper_bound = current_price * 1.3
    underlying_prices = np.linspace(lower_bound, upper_bound, 400)

    if option_type in ('long call', 'short call', 'long put', 'short put'):
        strike_price, premium = get_option_parameters()
        position = option_type.split()[0]
        print(f"Underlying Price: {current_price:.2f}, Strike Price: {strike_price:.2f}, Premium: {premium:.2f}, Position: {position.title
         ()} {option_type.split()[-1].title()}")

        if 'call' in option_type:
            payoffs = [call_payoff(price, strike_price, premium, position) for price in underlying_prices]
            title = f"{position.title()} Call Payoff at Expiration"
            break_even = strike_price + premium if position == 'long' else strike_price + premium
        else:
            payoffs = [put_payoff(price, strike_price, premium, position) for price in underlying_prices]
            title = f"{position.title()} Put Payoff at Expiration"
            break_even = strike_price - premium if position == 'long' else strike_price + premium

        plot_payoff(underlying_prices, payoffs, title)
        print(f"Break-even point: ${break_even:.2f}")
    else:
        print("Invalid option type selected.")
```

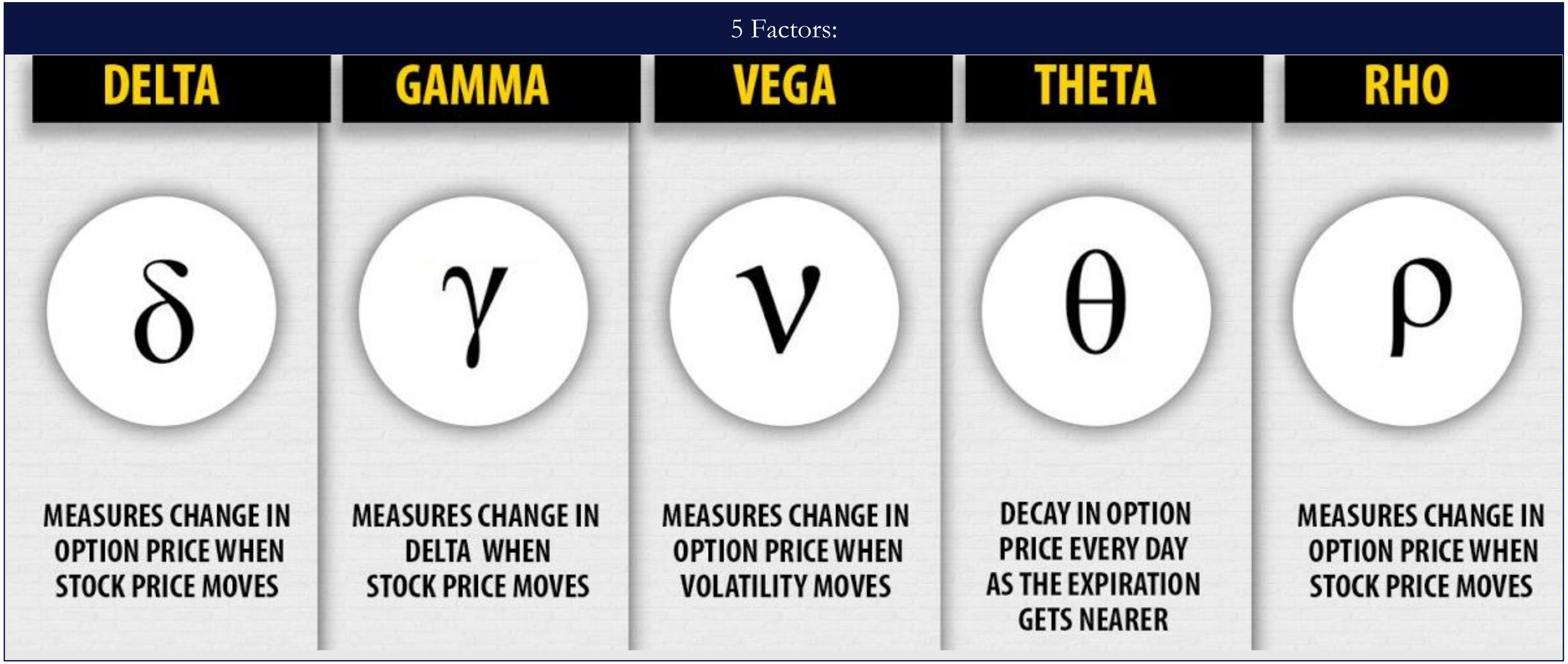
Output







Option Greeks

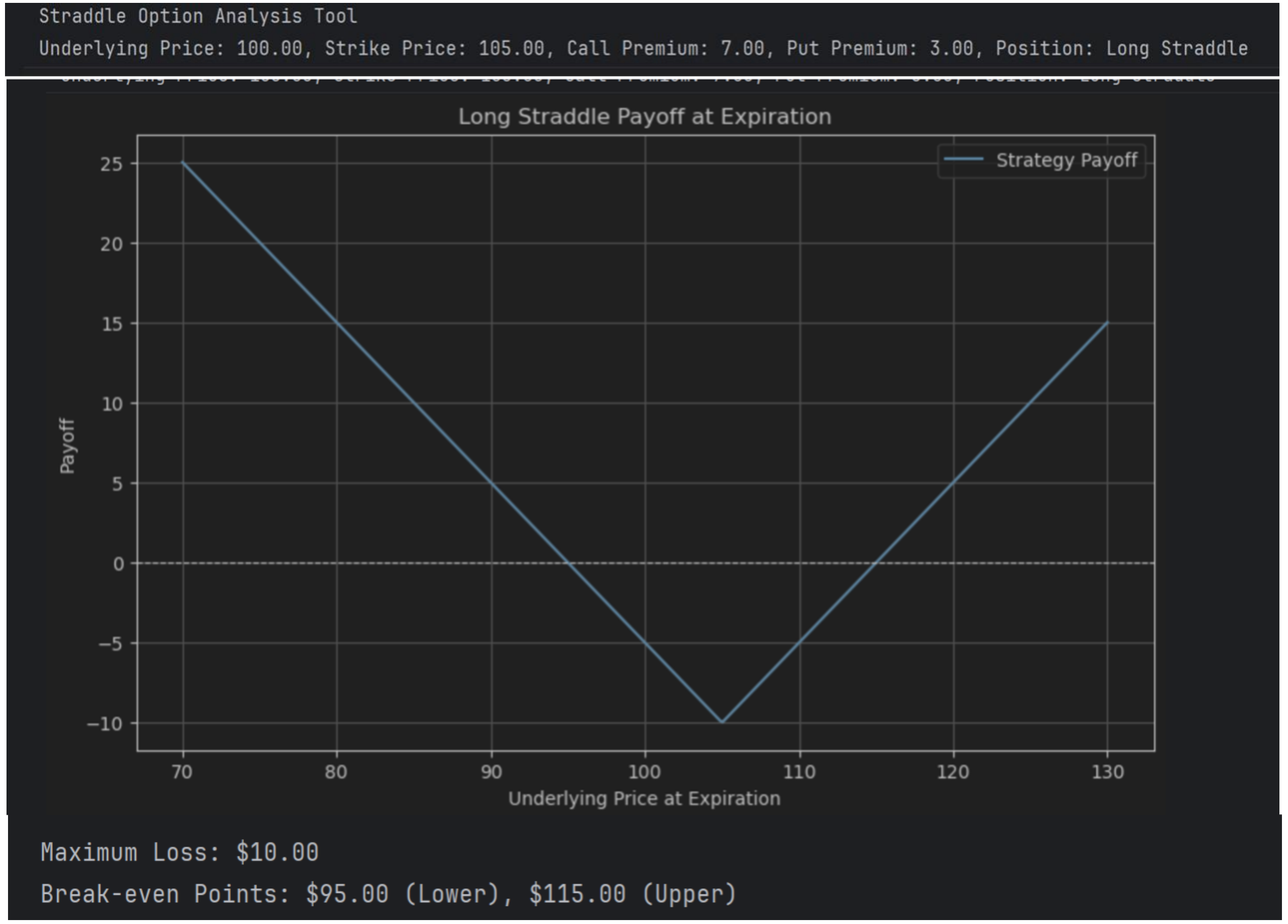


Payoff Tool Presentation

Long Straddle

Description

- Strategy: purchase long call and long put with the same expiration date and strike price, monitor the position and finally exit both legs
- Unlimited upside potential
- Max loss = both premiums paid
- Theta decay hurts position over time
- Benefits at higher IV levels



Payoff Tool Presentation

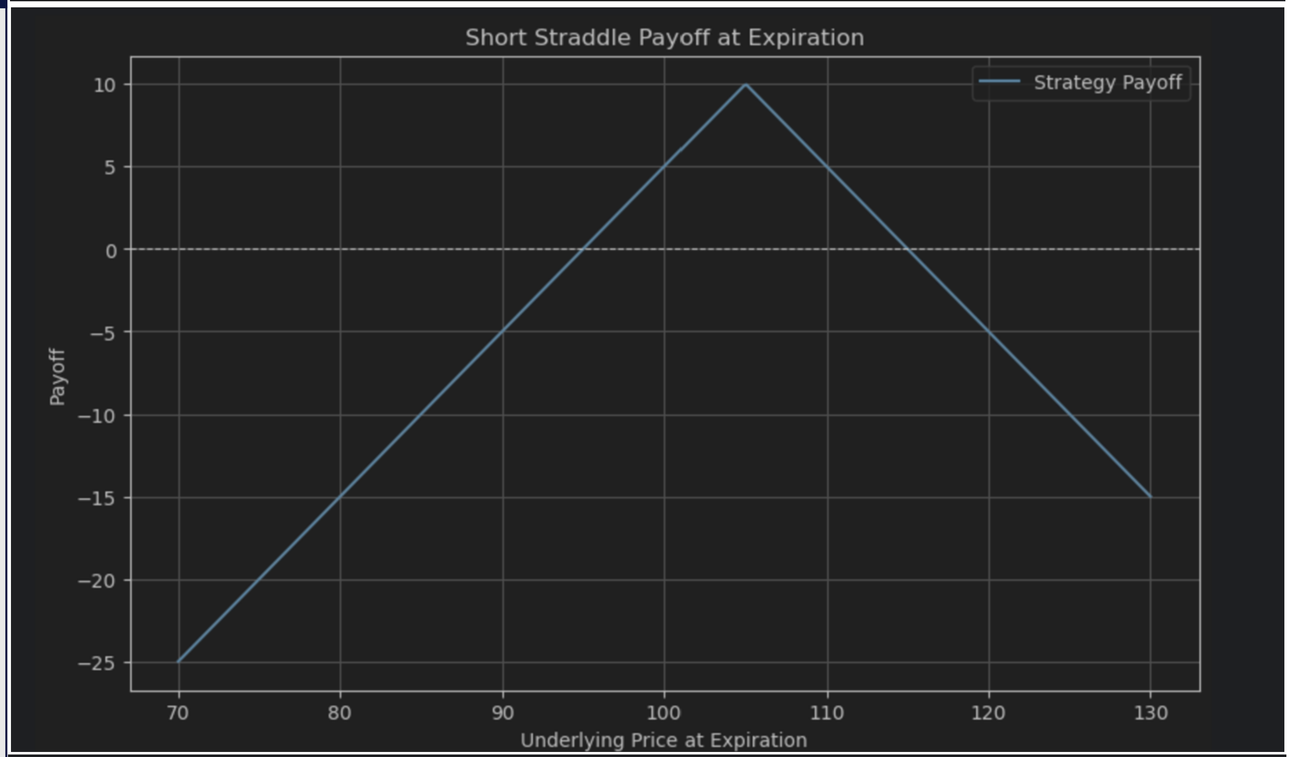
Short Straddle

Description

- Sell a call and a put with the same strike price and expiration
- Used when expecting low volatility and little price movement
- Max profit is the total premium received
- Max profit occurs if the stock finishes at the strike price
- Losses are unlimited if the stock rises sharply
- Significant losses if the stock drops sharply
- Breakevens: strike price ± total premium received
- High risk, limited reward
- Requires substantial margin due to risk

Straddle Option Analysis Tool

Underlying Price: 100.00, Strike Price: 105.00, Call Premium: 7.00, Put Premium: 3.00, Position: Short Straddle



Maximum Profit: $10.00

Break-even Points: $95.00 (Lower), $115.00 (Upper)

Code for Straddle

"straddle_payoff"

- Calculates the total profit or loss of a long or short straddle by summing the call and put payoffs based on the position type.

```
def straddle_payoff(underlying_price, call_strike, call_premium, put_strike, put_premium, position='long'):
    if position == 'long':
        call_payoff_long = call_payoff(underlying_price, call_strike, call_premium, 'long')
        put_payoff_long = put_payoff(underlying_price, put_strike, put_premium, 'long')
        return call_payoff_long + put_payoff_long
    elif position == 'short':
        call_payoff_short = call_payoff(underlying_price, call_strike, call_premium, 'short')
        put_payoff_short = put_payoff(underlying_price, put_strike, put_premium, 'short')
        return call_payoff_short + put_payoff_short
```

"get_straddle_parameters"

- Prompts the user for the strike price and premiums of call and put options, then returns these values as parameters.

```
def get_straddle_parameters(position):
    strike_price = float(input("Enter the strike price for both call and put options (e.g., 105.00): "))
    call_premium = float(input("Enter the call option premium (e.g., 7.00): "))
    put_premium = float(input("Enter the put option premium (e.g., 3.00): "))
    return strike_price, call_premium, put_premium
```

Setting Up Price Range

- Uses NumPy to ensure non-zero intrinsic value and generate a range of underlying prices based on user-defined parameters.

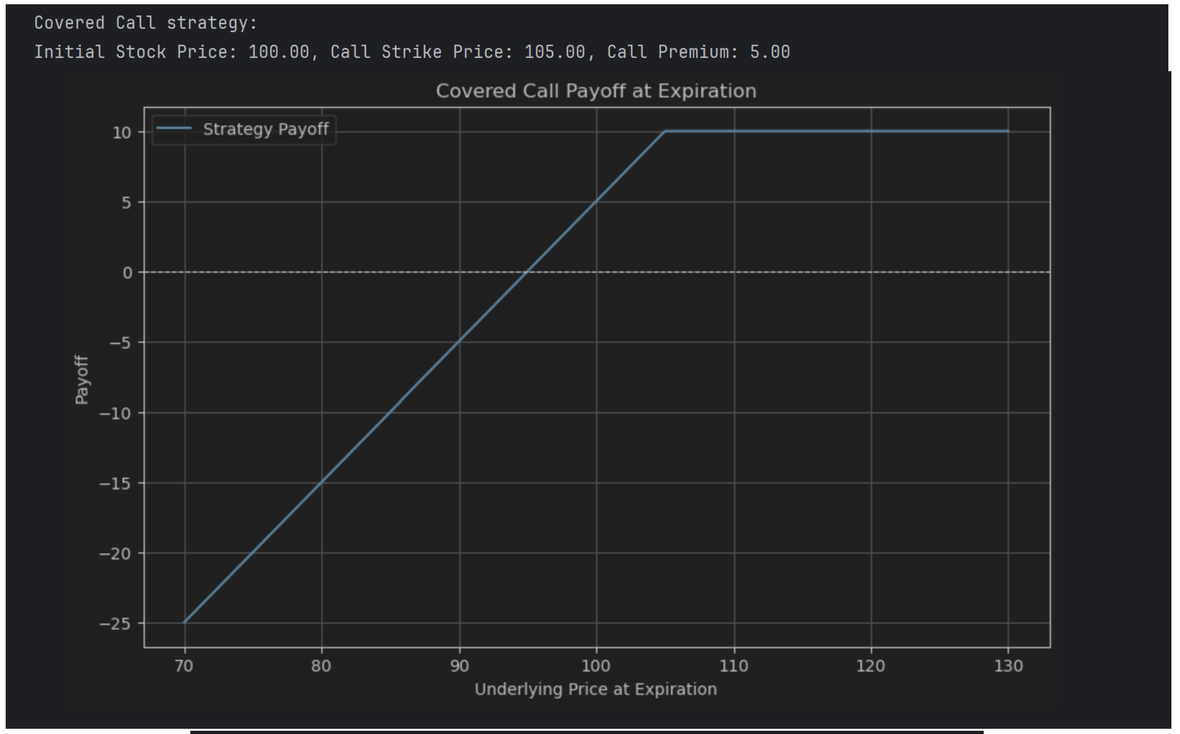
```
parameters = get_straddle_parameters(position_type)
if parameters:
    strike_price, call_premium, put_premium = parameters
    current_price = float(input("Enter the current underlying asset price (e.g., 100.00): "))
    lower_bound = max(0, current_price * 0.7)
    upper_bound = current_price * 1.3
    underlying_prices = np.linspace(lower_bound, upper_bound, 400)
```

Payoff Tool Presentation

Covered Call

Description

- Selling a call option
- Collect premium while limiting upside
  - Best if neutral or slightly bullish
- Cash from premium and protects the downside slightly
- Miss out on high profits if price surges – unlimited downside
- Low risk if stock tanks
- Can mitigate risk by owning the underlying asset



Break-even point for the Covered Call: $95.00

Covered Call Code

"covered_call_payoff"

- Calculates the net payoff of a covered call strategy by combining stock value change and short call payoff.

"get_covered_call_parameters"

- Retrieves user-defined parameters for the covered call strategy, including stock price, call strike price, and call premium.

Calculating and Visualizing Payoff

- Executes the covered call strategy, retrieves user parameters, calculates potential payoffs across a range of underlying prices, and plots the payoff.

```python
def covered_call_payoff(underlying_price, stock_price, call_strike, call_premium):
    stock_payoff = underlying_price - stock_price
    short_call_payoff = call_premium - np.maximum(0, underlying_price - call_strike)
    return stock_payoff + short_call_payoff
```

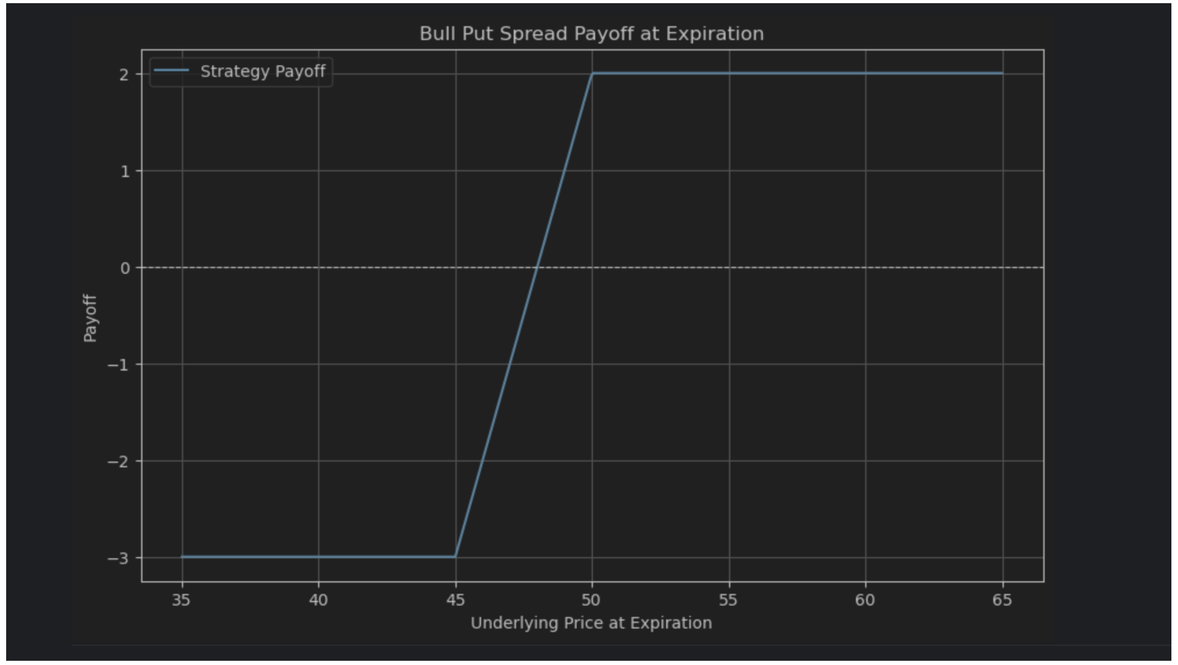
```python
def get_covered_call_parameters():
    stock_price = float(input("Enter the initial stock price (e.g., 100.00): "))
    call_strike = float(input("Enter the call option strike price (e.g., 105.00): "))
    call_premium = float(input("Enter the call option premium received (e.g., 5.00): "))
    return stock_price, call_strike, call_premium
```

```python
if __name__ == "__main__":
    print("Covered Call strategy:")
    stock_price, call_strike, call_premium = get_covered_call_parameters()
    lower_bound = max(0, stock_price * 0.7)
    upper_bound = stock_price * 1.3
    underlying_prices = np.linspace(lower_bound, upper_bound, 400)
    print(f"Initial Stock Price: {stock_price:.2f}, Call Strike Price: {call_strike:.2f}, Call Premium: {call_premium:.2f}")
    covered_call_payoffs = [covered_call_payoff(price, stock_price, call_strike, call_premium) for price in underlying_prices]
    plot_payoff(underlying_prices, covered_call_payoffs, 'Covered Call Payoff at Expiration')
    break_even = stock_price - call_premium
    print(f"\nBreak-even point for the Covered Call: ${break_even:.2f}")
```

Vertical Spread

Description

- Bull call (debit)
  - Buying a call and selling a call at a higher strike price
- Bear call (credit)
  - Selling a call and buying a call at a higher strike price
- Bull put (credit)
  - Selling a put and buying a put at a lower strike price
- Bear put (debit)
  - Buying a put and selling a put at a lower strike price
- Credit maximum loss is the difference in strike prices less net premium, Debit maximum loss is net premium
- Reduce premium cost (great during volatile markets)
- Hedges risk by taking both sides of a position



Code for Vertical Spread

Defining Spread Payoff Functions

- Defines four functions to calculate net payoffs for vertical spread strategies: bull call, bear put, bear call, and bull put spreads.
- Each strategy combines a long and short position on the same option type (call or put) at different strike prices.
- Uses call_payoff and put_payoff helper functions to calculate individual leg payoffs, accounting for premiums and option type.
- Returns the total strategy payoff at expiration by summing the long and short leg results.

```python
def bull_call_spread_payoff(underlying_price, lower_strike, higher_strike, lower_premium, higher_premium):
    long_call_payoff = call_payoff(underlying_price, lower_strike, lower_premium, 'long')
    short_call_payoff = call_payoff(underlying_price, higher_strike, higher_premium, 'short')
    return long_call_payoff + short_call_payoff
```

```python
def bear_put_spread_payoff(underlying_price, higher_strike, lower_strike, higher_premium, lower_premium):
    long_put_payoff = put_payoff(underlying_price, higher_strike, higher_premium, 'long')
    short_put_payoff = put_payoff(underlying_price, lower_strike, higher_premium, 'short')
    return long_put_payoff + short_put_payoff
```

```python
def bear_call_spread_payoff(underlying_price, lower_strike, higher_strike, lower_premium, higher_premium):
    short_call_payoff = call_payoff(underlying_price, lower_strike, lower_premium, 'short')
    long_call_payoff = call_payoff(underlying_price, higher_strike, higher_premium, 'long')
    return short_call_payoff + long_call_payoff
```

```python
def bull_put_spread_payoff(underlying_price, higher_strike, lower_strike, higher_premium, lower_premium):
    short_put_payoff = put_payoff(underlying_price, higher_strike, higher_premium, 'short')
    long_put_payoff = put_payoff(underlying_price, lower_strike, lower_premium, 'long')
    return short_put_payoff + long_put_payoff
```

Code for Vertical Spread

"get_vertical_spread_parameters"

- Gathers key spread parameters by prompting the user for strike prices and premiums based on the selected strategy
- Provides contextual input prompts using if/elif logic tailored to bull call, bear put, bear call, or bull put spreads
- Ensures numerical format by converting inputs to floats for accurate calculations
- Returns a structured output as a tuple for use in payoff analysis functions

```python
def get_vertical_spread_parameters(spread_type):
    if spread_type == 'bull call':
        lower_strike = float(input("Enter the lower strike price (e.g., 45.00): "))
        higher_strike = float(input("Enter the higher strike price (e.g., 50.00): "))
        lower_premium = float(input("Enter the premium paid for the lower strike call (e.g., 3.00): "))
        higher_premium = float(input("Enter the premium received for the higher strike call (e.g., 1.00): "))
        return lower_strike, higher_strike, lower_premium, higher_premium
    elif spread_type == 'bear put':
        higher_strike = float(input("Enter the higher strike price (e.g., 55.00): "))
        lower_strike = float(input("Enter the lower strike price (e.g., 50.00): "))
        higher_premium = float(input("Enter the premium paid for the higher strike put (e.g., 2.00): "))
        lower_premium = float(input("Enter the premium received for the lower strike put (e.g., 0.50): "))
        return higher_strike, lower_strike, higher_premium, lower_premium
    elif spread_type == 'bear call':
        lower_strike = float(input("Enter the lower strike price (e.g., 50.00): "))
        higher_strike = float(input("Enter the higher strike price (e.g., 55.00): "))
        lower_premium = float(input("Enter the premium received for the lower strike call (e.g., 2.00): "))
        higher_premium = float(input("Enter the premium paid for the higher strike call (e.g., 0.50): "))
        return lower_strike, higher_strike, lower_premium, higher_premium
    elif spread_type == 'bull put':
        higher_strike = float(input("Enter the higher strike price (e.g., 50.00): "))
        lower_strike = float(input("Enter the lower strike price (e.g., 45.00): "))
        higher_premium = float(input("Enter the premium received for the higher strike put (e.g., 3.00): "))
        lower_premium = float(input("Enter the premium paid for the lower strike put (e.g., 1.00): "))
        return higher_strike, lower_strike, higher_premium, lower_premium
```

Code for Vertical Spread

Generate Price Range

- Prompts user for current price of the asset.
- Calculates the price range (70% to 130% of the current price).
- Generates 400 price points within that range using NumPy's linspace.

```
    if parameters:
        current_price = float(input("Enter the current underlying asset price (e.g., 50.00): "))
        lower_bound = max(0, current_price * 0.7)
        upper_bound = current_price * 1.3
        underlying_prices = np.linspace(lower_bound, upper_bound, 400)
```

Spread Payoff Calculations

- Calculates and displays option parameters based on the selected spread type.
- Computes payoff for each underlying price and generates a payoff plot.
- Displays maximum profit, loss, and break-even point.

```
if spread_type == 'bull call':
    lower_strike, higher_strike, lower_premium, higher_premium = parameters
    print(f"Underlying Price: {current_price:.2f}, Lower Strike Price: {lower_strike:.2f}, Higher Strike Price:
     {higher_strike:.2f}, Lower Premium: {lower_premium:.2f}, Higher Premium: {higher_premium:.2f}, Spread Type: Bull
     Call")
    spread_payoffs = [bull_call_spread_payoff(price, lower_strike, higher_strike, lower_premium, higher_premium) for
     price in underlying_prices]
    plot_payoff(underlying_prices, spread_payoffs, 'Bull Call Spread Payoff at Expiration')
    max_profit = (higher_strike - lower_strike) - (lower_premium - higher_premium)
    max_loss = (lower_premium - higher_premium)
    print(f"\nMaximum Profit: ${max_profit:.2f}")
    print(f"Maximum Loss: ${max_loss:.2f}")
    print(f"Break-even Point: ${lower_strike + (lower_premium - higher_premium):.2f}")
```

Limitations & Improvements

Description

- Purpose of the Model
  - Illustrates possible outcomes, not intended to be used as a predictive tool
  - Aimed to help understand rather than forecast market behavior
- Visualizations and Modeling
  - Effectively visualized potential profits based on price changes in the underlying security
  - Helped understand the return in correlation to the risk
- Limitations
  - Cannot make probability-based predictions
- Improvements
  - The models could be streamlined by consolidating functions and inputs into one code, enabling strategy selection without switching sections and reducing redundancy.

Traders@SMU

Q&A

Sources

- https://www.investopedia.com/terms/c/calloption.asp
- https://www.investopedia.com/terms/p/putoption.asp
- https://www.investopedia.com/terms/c/coveredcall.asp
- https://www.tastylive.com/concepts-strategies/vertical-spread
- https://www.investopedia.com/terms/s/straddle.asp
- https://www.euronext.com/en/news/what-long-straddle
- https://www.pyquantnews.com/topics/options-trading-with-python